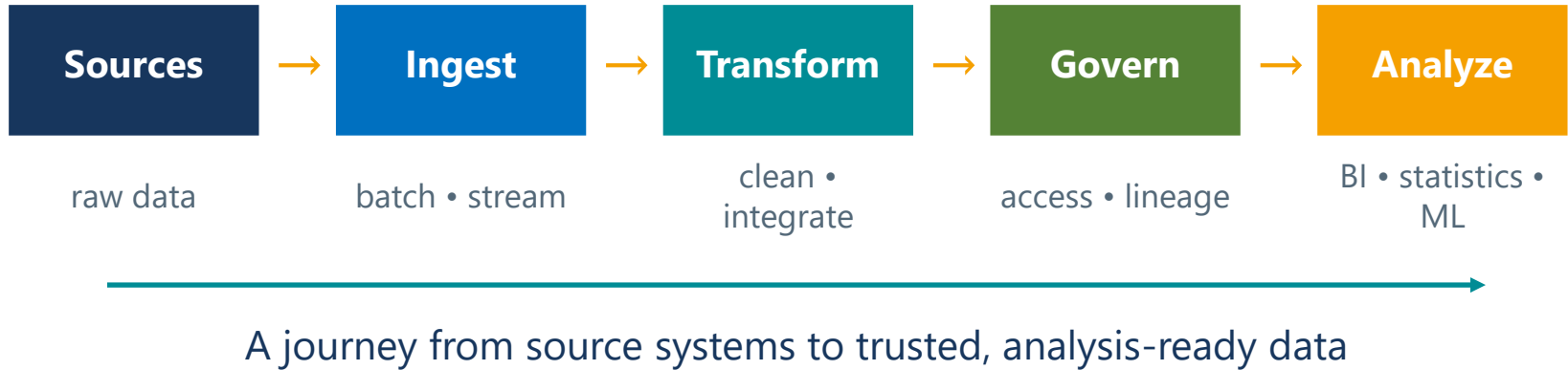




Data Engineering: Ingestion, Transformation and Lakehouse Design

Outline

1



Data Lakehouse

Source systems → Ingested data → governed, reliable, analysis-ready data

2



Data Pipeline Design Decisions

Batch vs. streaming • Push vs. pull • Full vs. incremental/CDC

3



Medallion Architecture

Bronze → Silver → Gold layers

4



Data Processing using SQL

Clean, standardize, validate, and enrich

5



Data Processing using DataFrame

Clean, standardize, validate, and enrich



Data Lakehouse

OLTP Captures Business Activity; OLAP Turns It into Insight

	OLTP — operational workloads	OLAP — analytical workloads
Purpose:	record day-to-day transactions.	find patterns and answer questions.
Typical action:	insert or update a few rows.	scan and summarize many rows.
Optimized for:	fast, reliable writes with low latency.	joins, trends, and aggregates.
Example:	place an order and update stock.	compare monthly sales by region.

Why separate them?

Prevent analytical workloads from slowing down operational transactions.

The business relies on both workloads: one supports daily operations, while the other enables analysis and insight

Data Warehouse vs Data Lake vs Data Lakehouse

DATA WAREHOUSE

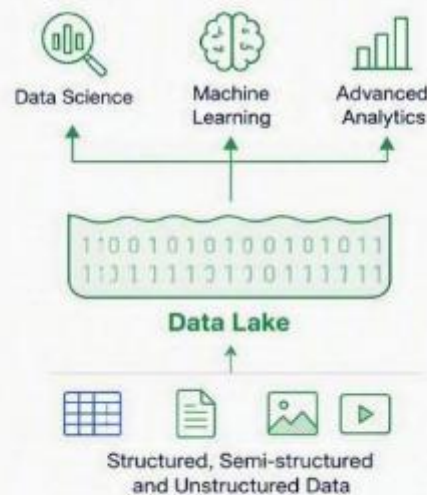
Structured data for reporting and business intelligence



	Data Type	Structured
	Schema	Schema-on-Write (Rigid)
	Performance	Fast for complex queries and aggregations
	Data Quality	High (governed, validated)
	Best For	Reporting, BI, dashboards, KPIs

DATA LAKE

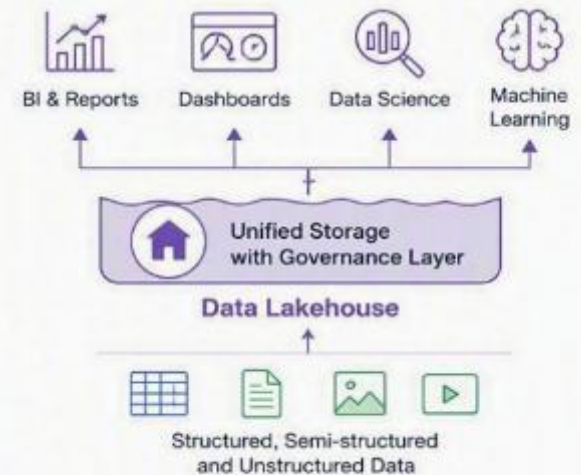
All raw data for flexibility and advanced analytics



	Data Type	Structured, semi-structured and unstructured
	Schema	Schema-on-Read (Flexible)
	Performance	Slower for complex queries (scan large volumes)
	Data Quality	Variable (raw data)
	Best For	Data science, ML, exploration, real-time data

DATA LAKEHOUSE

Best of both worlds: flexibility of a lake with the performance and governance of a warehouse



	Data Type	Structured, semi-structured and unstructured
	Schema	Schema-on-Read and Schema-on-Write
	Performance	Fast and scalable for BI, ML and analytics
	Data Quality	High (governed + flexible)
	Best For	BI, data science, ML, real-time analytics



Key Takeaway: Use a **Data Warehouse** for trusted reporting, a **Data Lake** for flexibility and exploration, and a **Data Lakehouse** when you need both performance and flexibility together.

Popular Lakehouse Platforms - Four Ways to Run Analytical Workloads

	 Databricks	 Snowflake	 Microsoft Fabric	 DuckDB
WHAT IT IS	Clusters of machines running your code	A managed engine you just write SQL on	An all-in-one analytics platform	A tiny engine running inside your program
STRENGTH	Big data pipelines, streaming and ML	Fast, reliable SQL for many concurrent users	Reports and dashboards with little setup	Instant queries on files, with no setup
YOU PAY FOR	Time the cluster is switched on	Seconds of query time; idle is free	Prepaid capacity shared by all tools	Free — just your laptop or a server
USE IT WHEN	Large-scale data engineering	Analysts need dependable SQL, not servers	Organizations already using the Microsoft ecosystem	Data fits on one machine and you iterate fast



How to choose

- Start with one machine until the data outgrows its memory.
- Bigger platforms add scale, sharing and security at a higher cost.

Two Ideas Behind the Modern Lakehouse

Storage and Compute Are Separate

WHAT IT IS

Data sits in cheap cloud storage; engines connect to it when needed

WHY IT HELPS

Add or remove processing power without copying or reloading data

IN PRACTICE

Each team runs its own engine on the same data, then switches it off

WATCH OUT

How the files are organized now decides how fast queries run

Open Table Formats

WHAT IT IS

A metadata layer that turns plain files into proper tables

WHY IT HELPS

Safe updates and deletes, schema changes, and a view of older versions

IN PRACTICE

Roll back a bad load, or reproduce yesterday's numbers

WATCH OUT

Tables need housekeeping: merge small files, clear old versions

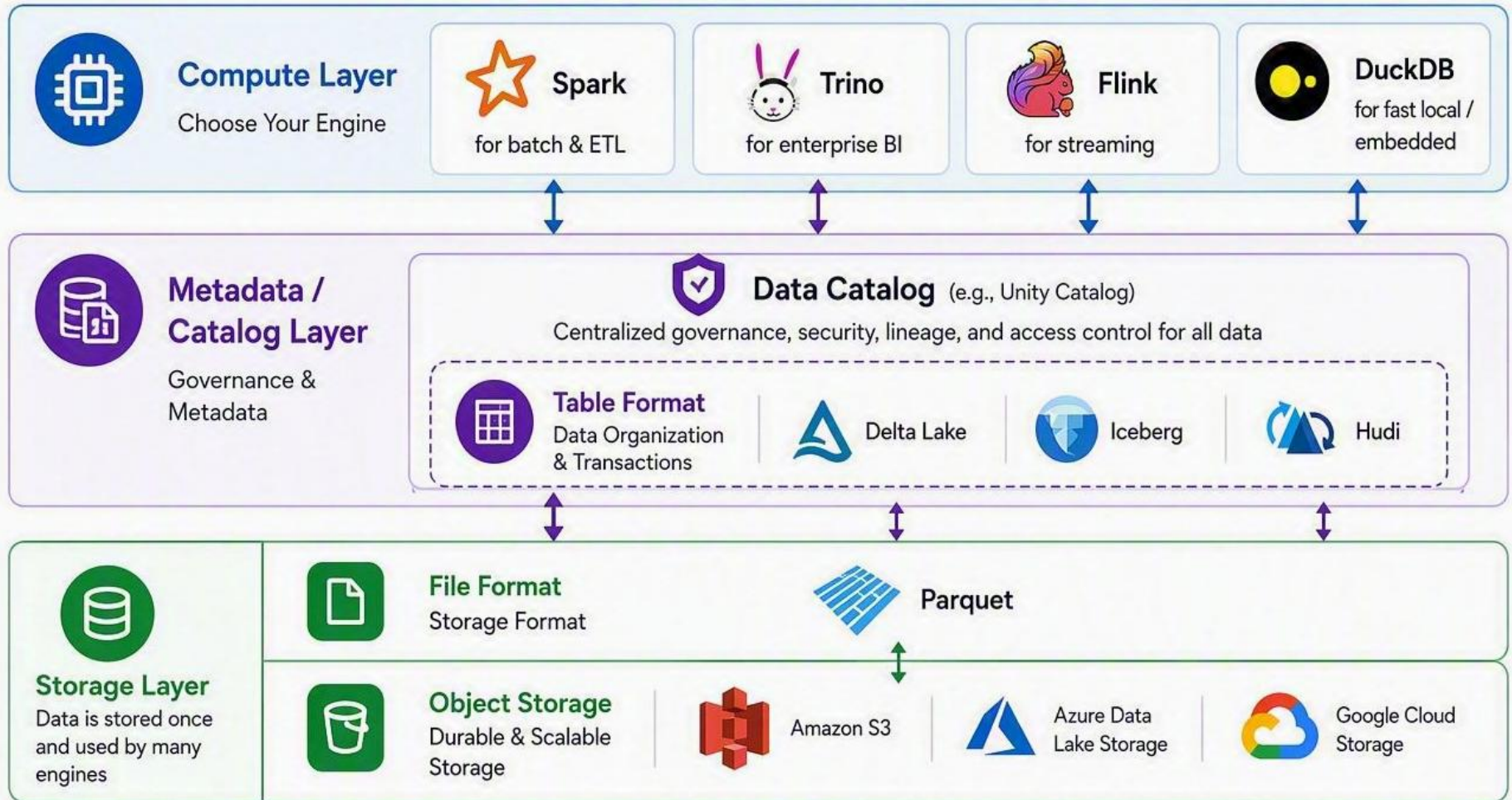


Why they work together

- Cheap storage keeps one copy of the data; open tables make changing it safe.
- Any engine can then read the same trusted tables, without moving or copying them.

Lakehouse Architecture

The Separation of Storage, Compute, and Centralized Governance Layers



How the Three Layers Work Together

Compute Layer

Engines that read and process the data

Start one for a job and shut it down when it finishes; run several at once, on the same data.

For example: Spark, Trino, Flink or DuckDB

Metadata & Catalog

The layer that makes files behave like tables

Tracks which files form a table, its columns, its versions, and who is allowed to read it.

For example: a table format such as Iceberg, plus a catalog

Storage Layer

One shared copy of the data

Columnar files sitting in cloud storage — cheap, and charged separately from processing.

For example: Parquet files in S3, Azure Data Lake or Google Cloud Storage



How one query runs

1. The engine asks the catalog which files belong to the table.
2. It reads only those files straight from storage — nothing is copied in.
3. So you can add or swap an engine without moving any data.

The end-to-end data journey



✓ Evidence preserved

- Identify where data originates
- Raw records + source identifiers, arrival metadata
- Move new or changed data
- E.g. Checkout database, product catalog, inventory events

✓ Meaning clarified

- Clean, standardize, join, and apply business definitions
- E.g., Fix dates, join product data, calculate the line total
- Test rules and monitor quality
- E.g., Order ID present; quantity > 0

✓ Use governed

- Define data ownership and access controls
E.g., only the Finance team can access revenue data.
- Publish trusted data for use
- E.g., Curated orders feed a dashboard and stock alert

Governance keeps data secure, findable and traceable

Capability	Practical question	Online-shop example
Data cataloging	What is it, who owns it and when is it refreshed?	daily_product_sales has a definition, owner and daily refresh time.
Access control	Who can view, change or share it?	Analysts see sales; only Finance can access payment details.
Data lineage	Where did it come from and what transformed it?	Orders API → Bronze → Silver orders → daily_product_sales.
Monitoring and auditing	Is it healthy, and who used or changed it and when?	Alert on late loads; log access and permission changes.

Good governance helps people find the right data, trust it, and use it safely.



Data Pipeline Design Decisions

Agree these five things with the source owner

Agree with the source owner	What to decide	What we agreed (online orders)
1 Contact and support	Who owns the source, who is on call to resolves issues	Commerce team owns sales data
2 Grain, schema, and key	What one row represent (grain), the schema of each entity, and the key that uniquely identifies it	One row = one item in an order; key = order_id + line_id; 12 agreed fields
3 Full data or incremental data	Do we get the whole table each time, or only new, updated and deleted records	Hourly incremental file of new and changed orders; cancellations sent with a change flag
4 Delivery and Ingestion frequency	How the data reaches us and how often	Orders file provided hourly
5 Data quality and recovery	What to do with bad rows	Set aside orders with no ID

Choose the right data grain

Choose	Use it when	Online-orders example
Detailed grain	Supports drill-down and root-cause analysis.	Need order items for basket-level analysis. One row per order item Key = (<i>order_id</i> , <i>line_id</i>)
Summary grain	Used for KPIs, dashboards, trends, forecasting, or executive reporting	One row per day Fields = date, sales_amount, order_count, avg_order_value
Granularity Best Practice	Keep detailed and aggregated data in separate tables. Create summaries from detailed data through ETL/ELT pipelines.	Order Items → Daily Sales → Monthly Sales

- **Data grain = what a single row represents**
- Define the grain before designing the table
- Keep one consistent grain per table
- Store detailed data first; create summaries separately

How data moves: batch vs streaming ingestion

Ask: how fast must we act, and who should start the movement?

Compare	Scheduled batch (pull)	Continuous streaming (push)
Who starts it	The pipeline asks the source on a schedule	The source sends each event the moment it happens
How fresh	Minutes to hours old, ready at a fixed time	Seconds-to-minutes, new events arrive continuously
How it works	One job reads and process everything changed since the last run	Each event is handled on arrival, one at a time
Online-order example	At 2 a.m. pull all of yesterday's orders for the sales database	Block a suspicious card payment before it goes through
Choose it when	The decision can wait for the next run. Ideal for historical analysis and reporting	Waiting for the next run costs money or trust. Ideal for IoT monitoring, financial transaction alerts, and supply chain tracking.
Watch for	Missed changes and heavy load on the source system	Duplicate and out-of-order events, and replay after an outage
Cost and effort	Cheaper, and a failed run can simply be run again	Always on, needs monitoring, alerting and on-call

Can the decision wait? Yes → scheduled batch (pull) | No → streaming (push)

Both need a plan for failures, retries and duplicate data.

How much to load: Full, incremental or CDC

Ask: Do we reload everything, or only what changed?

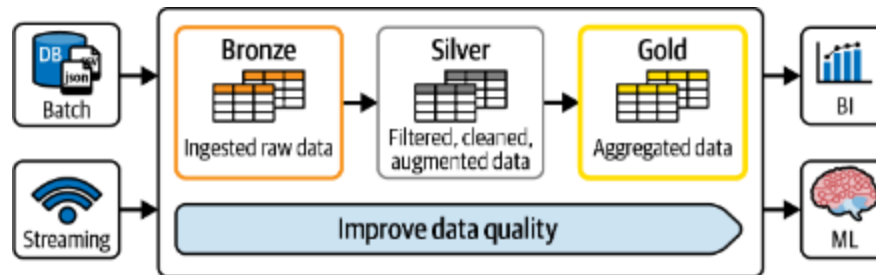
Option	What it means	Use it when	Online-order example
Full load	Copy the whole table again	First load, or the table is small enough to copy quickly	Copy the whole product catalogue again each night
Incremental load	Copy only rows added or changed since last time	Too big to reload, and rows carry a reliable date or ID	Load only the orders placed since last night's run
Change Data Capture (CDC)	Stream every add, change, and delete	Data copies must stay in sync in near real time	Follow one order as it is placed, paid, or cancelled

First time do Full load then Incremental periodically. CDC only when needed.

Always record what was loaded and when.



Medallion Architecture



Medallion Architecture



- A Medallion architecture arranges data into three layers, enhancing the data's structure and quality as it progresses through the layers

Bronze → Silver → Gold: data gets cleaner at each stage

Bronze

Keep it

Stored exactly as it
arrived

Silver

Fix it

Cleaned and de-
duplicated

Gold

Use it

Summarised for one
question

Online-order example

**“raw order files
dropped by each store”**

**one clean orders table,
one row per order line**

**daily sales by product,
ready for the
dashboard**

Raw → Clean → Useful

If processing fails, replay from Bronze to rebuild Silver and Gold without asking the source to resend everything.

Bronze: the row plus tracking columns

15

Ingestion details added to every row such as:	Online-order example	Why it matters
source_name	orders_api	Says which system the row came from
ingested_at	2026-09-03 10:03	Shows when it arrived -> can help identify late data
load_id	batch_2026_09_03_01	Lets you replay just one bad run

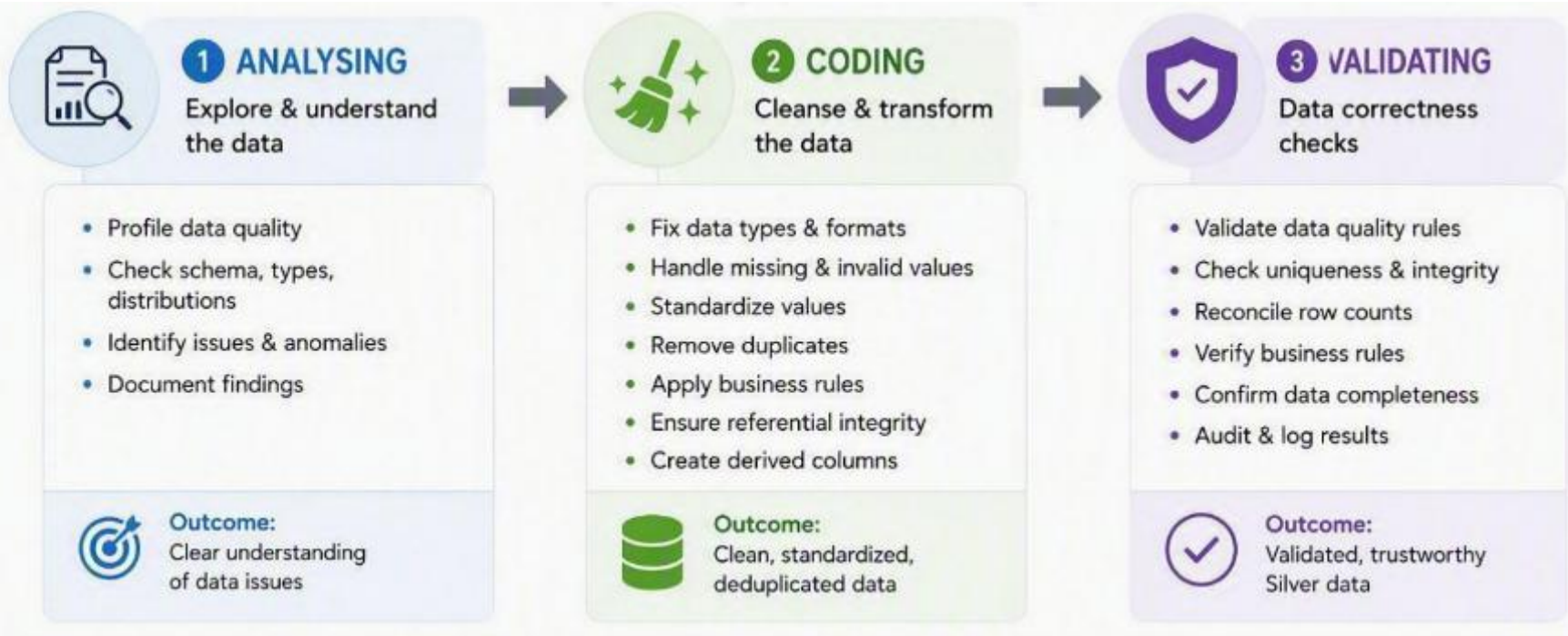
Bronze can be messy, but the original values are never changed.

Write Bronze once, never edit it. Fix data in Silver, so you can always rebuild.



Building the Silver Layer

Clean, standardize, validate, and enrich raw data.



Silver Layer Data Transformations

Transformation	What it means	Online-order example
Fix data types	Text becomes numbers, dates and Booleans	"19.99" becomes 19.99; "2026-09-03" becomes a date
Handle missing values	Fill, flag, or quarantine blanks	Order with no customer_id goes to quarantine
Remove duplicates	Keep one row per unique key	Same order sent twice by the API: keep the latest
Standardise values	One spelling, one unit, one code, one format	"QA", "Qatar", "qatar" all become QAT; dates stored as YYYY-MM-DD
Apply business rules	Reject rows that cannot be true	quantity <= 0 or a future order_date is rejected
Integrate reference data	Connect each row to its related data	Order joined to its customer; product_id 88 gains its name
Enrich with new fields	Add a useful value calculated from existing fields	delivery_days = delivered_at – order_date

**Silver is where messy data becomes data users can trust.
Not every dataset needs every step.**

Silver in practice: a Pandas example

17

Step	Pandas code	What it does
Load Bronze	<code>df = pd.read_csv("bronze/orders")</code>	Reads the raw rows as received
Fix numbers	<code>df["price"] = pd.to_numeric(df["price"])</code>	Bad prices become NaN, not a crash
Fix dates	<code>df["date"] = pd.to_datetime(df["date"])</code>	Text dates become real dates
De-duplicate	<code>df = df.drop_duplicates(["order_id", "line_id"])</code>	One row per order line
Validate	<code>df = df[df["quantity"] > 0]</code>	Removes impossible quantities
Join reference	<code>df = df.merge(products, on="product_id")</code>	Adds product name and category

Same four moves every time: load, fix, de-duplicate, validate.

Silver rules: what to do when data fails

18

Rule	Why it exists	What happens when it fails
order_id is not null	Without a key, the row cannot be traced	Row moves to quarantine for the source team to investigate and correct
quantity > 0	A sale of zero or minus items is not real	Row is rejected and not counted in the daily report
price is a number	Text prices break every calculation	Value set to NaN, row flagged for review
order_date <= today	Future dates mean a broken clock or bad parse	Row is held back until the source is fixed
product_id exists in products	Unknown products break sales reporting	Row is kept but marked "unknown product"

Never delete a bad row silently - always record it.

A rejected row is NOT discarded data as it may contain valuable information.



Building the Gold Layer

Create business metrics, summaries, and insights from trusted Silver data.



Gold Layer Build Steps



Gold Layer Data Transformations

Transformation	What it means	Online-order example
Aggregate to a grain	Roll many rows up to the level the business asks about	Aggregate individual order lines into units sold and revenue per product, per store, per day.
Derive business metrics	Turn raw columns into the agreed measures	$\text{revenue} = \text{quantity} \times \text{unit_price} - \text{discount}$
Model facts and dimensions	One table of events joined to descriptive tables	fact_sales joined to dim_product, dim_store and dim_date
Add time comparisons	Pre-compute the periods people always ask for	Month-to-date revenue and the same day last year
Rename for the business	Technical names become words users recognise	qty_ord becomes Units sold; amt_net becomes Revenue
Filter to the audience	Publish only the rows and columns a user may see	A store manager sees only their own store's rows

Gold is where data becomes an answer people can act on.

Gold in practice: a Pandas example

Step	Pandas code	What it does
Load Silver	<code>df = pd.read_parquet("silver/orders")</code>	Start from clean, trusted rows
Add the measure	<code>df["revenue"] = df.qty * df.price</code>	Apply the agreed revenue definition
Set the grain	<code>df["day"] = df.order_ts.dt.date</code>	Cut timestamps down to a single day
Group	<code>g = df.groupby(["day", "store", "product"])</code>	One row per product, store and day
Aggregate	<code>gold = g.agg(units=("qty", "sum"))</code>	Sum the measures for each group
Publish	<code>gold.to_parquet("gold/daily_sales")</code>	Save the table dashboards read

Gold is a grain, a measure, and a table people can query.

Gold rules: keep numbers trustworthy

20

Rule	Why it exists	What happens when it breaks
One definition per metric	Revenue must mean the same thing everywhere	Two dashboards disagree and people stop trusting both
Build only on Silver	Gold never reaches back into raw Bronze	Unclean rows leak straight into reported numbers
Rebuild, never patch	Any day can be recomputed from Silver	Hand fixes cannot be repeated or audited later
Check totals every run	Gold revenue must match Silver revenue	The run is held back and the team is alerted
Late data reopens the day	Orders can still arrive after midnight	Yesterday's totals are recomputed, not quietly ignored
Document every column	Users must know exactly what a column means	People invent their own meaning and report it

Trust is built by rules that run the same way every time.



Data Processing using SQL

SQL Runs in a Logical Order — Not the Order You Write

Logical execution order

1 FROM — pick the tables



2 WHERE — filter rows



3 GROUP BY — build groups



4 HAVING — filter groups



5 SELECT — choose columns



6 ORDER BY — sort output

The order you write it

```
SELECT country, COUNT(*) AS orders
FROM sales.orders
WHERE order_date >= '2026-01-01'
GROUP BY country
HAVING COUNT(*) > 100
ORDER BY orders DESC
LIMIT 10;
```

Why it matters: WHERE filters rows before grouping; HAVING filters after

Joins Combine Tables

	What you get	Use it when
INNER	Only rows matching in both tables	Every order must have a valid customer
LEFT	All left rows; NULLs where no match	Keep all customers, even those with no orders
RIGHT	All right rows; NULLs where no match	Rarely used — rewrite as LEFT for readability

```
SELECT c.customer_name,  
       COUNT(o.order_id) AS orders  
FROM sales.customers AS c  
LEFT JOIN sales.orders AS o  
      ON o.customer_id = c.customer_id  
GROUP BY c.customer_name;
```

Common trap

A LEFT JOIN plus a WHERE filter on the right table silently becomes an INNER JOIN.

Put that condition in the ON clause instead.



Aggregation Turns Raw Rows Into Business Metrics

BEFORE — raw order rows

order_id	country	amount
1001	Qatar	120.00
1002	Qatar	80.00
1003	UAE	200.00
1004	UAE	50.00
1005	Qatar	45.00

The query

```
SELECT country,  
       COUNT(*) AS orders,  
       SUM(amount) AS revenue,  
       AVG(amount) AS avg_order  
FROM sales.orders  
GROUP BY country  
HAVING SUM(amount) > 100  
ORDER BY revenue DESC;
```

AFTER — one row per group

country	orders	revenue	avg_order
Qatar	3	245.00	81.67
UAE	2	250.00	125.00



Important Notes

COUNT(*) counts rows; COUNT(col) skips NULLs.

Every non-aggregated SELECT column must appear in GROUP BY.

HAVING filters groups; WHERE filters rows before grouping.



CTEs Turn Sub Query Into Readable Steps

BEFORE — nested subquery

```
SELECT customer_id, COUNT(*) AS cnt
FROM (
  SELECT *
  FROM sales.orders
  WHERE order_date >=
    CURRENT_DATE - INTERVAL '30' DAY
) AS t
GROUP BY customer_id;
```

AFTER — named CTE

```
WITH recent_orders AS (
  SELECT *
  FROM sales.orders
  WHERE order_date >=
    CURRENT_DATE - INTERVAL '30' DAY
)
SELECT customer_id, COUNT(*) AS cnt
FROM recent_orders
GROUP BY customer_id;
```



Readable each step gets a name



Reusable reference a CTE more than once



Debuggable run one step at a time



Teaching points

A Common Table Expression (CTE) is a name for a result set, not a stored table — it disappears when the query ends.



Production-Ready SQL: Habits That Scale

Habits that survive code review

Filter early push WHERE before joins to shrink the scan

Alias with meaning sales.orders AS o, not AS a

Qualify schemas sales.orders, never a bare orders

Format consistently UPPERCASE keywords, lowercase identifiers

Select real columns SELECT * breaks silently when schemas drift

Mind NULLs NULL = NULL is unknown; use IS NULL

The four SQL sub-languages

Family	Commands	Purpose
DDL (Data Definition Language)	CREATE, ALTER, DROP	define structure
DML (Data Manipulation Language)	SELECT, INSERT, UPDATE, DELETE	work with data
DCL (Data Control Language)	GRANT, REVOKE	control access
TCL (Transaction Control Language)	COMMIT, ROLLBACK	manage transactions



Where SQL fits the pipeline

Bronze: land raw data as it arrived.

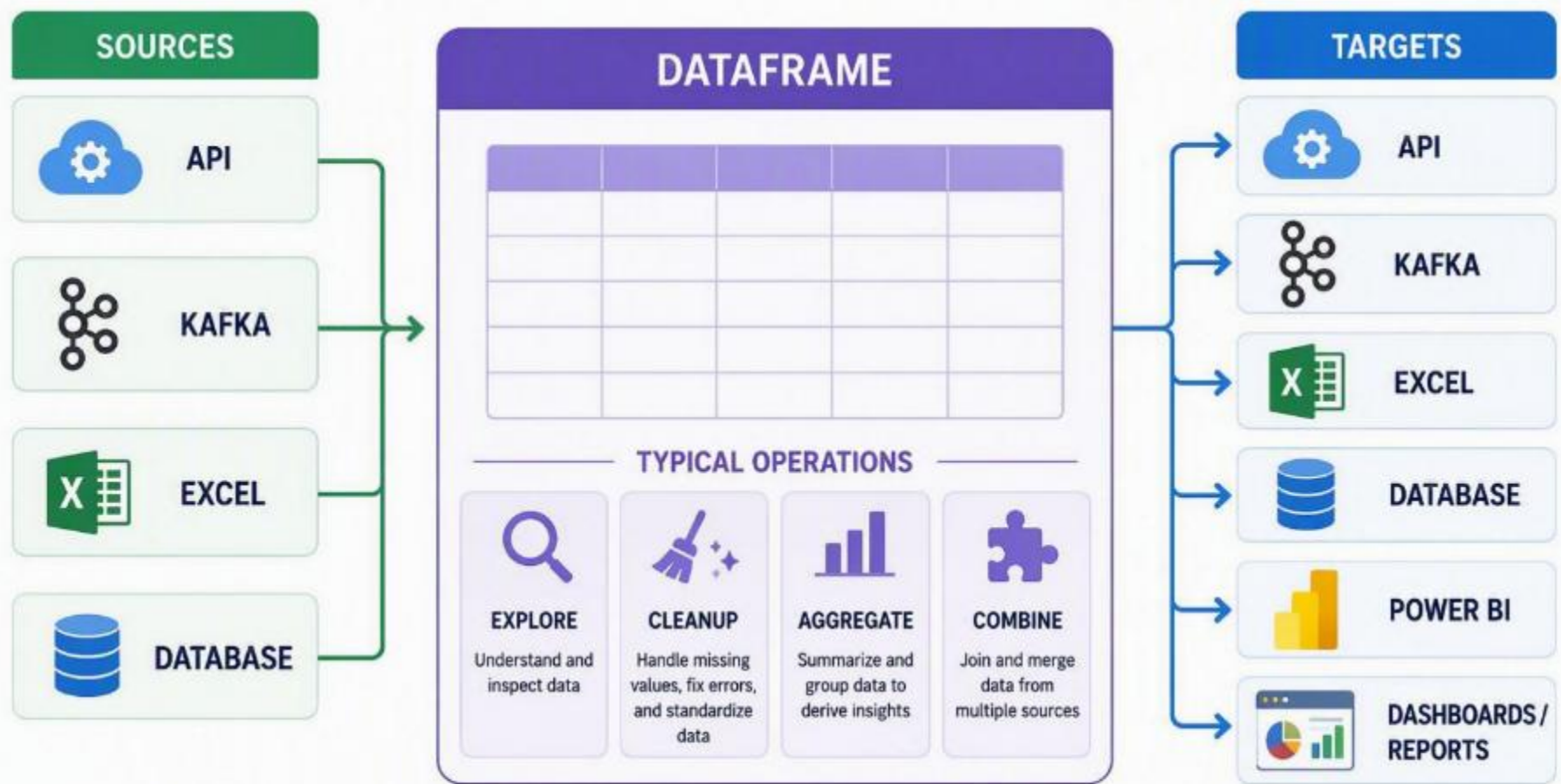
Silver: clean, deduplicate, and conform types with SQL transforms.

Gold: aggregate into the tables analysts and models actually query.



Data Processing using DataFrame

DataFrame = Core of ETL and Analytics



ONE STRUCTURE. MANY PURPOSES. DataFrame connects diverse sources to multiple targets through consistent operations—powering both data engineering and analytics.

Pandas, Polars, PySpark: Same DataFrame, Different Engine

pandas

BEST FOR

Exploration, CSV cleanup, small ad-hoc analysis

SWEET SPOT

< 5M rows, < 500 MB files

STRENGTH

Simple, flexible, huge ecosystem

LIMIT

Single-threaded, memory-bound, eager only

Polars

BEST FOR

Mid-size data on one machine, CPU-heavy work

SWEET SPOT

5M–50M rows, multi-GB CSV/Parquet

STRENGTH

Multi-core, lazy evaluation, Arrow-native

LIMIT

Single machine — not distributed

PySpark

BEST FOR

Large pipelines, governed production data, big joins

SWEET SPOT

> 50M rows, distributed Parquet/Delta

STRENGTH

Distributed, Delta + Unity Catalog, schema enforcement

LIMIT

Cluster overhead on small jobs; infrastructure to set up



Choosing between them

Rough guide on 10M rows: pandas ~40s • Polars ~5s on the same laptop • Spark 8–12s locally, but scales linearly past 100M rows.

A DataFrame brings a table into Python memory

Think of it as a Python table: labeled rows, named columns, and mixed data types.

From file to dataframe

orders.csv → read_csv → DataFrame

Index = row labels • Columns = fields

index	order_id	channel	amount	status
0	1041	Web	120	Complete
1	1042	Store	85	Pending
2	1043	App	60	Complete

Load, inspect, select, filter

```
import pandas as pd

orders = pd.read_csv(
    "orders.csv")

orders.head()
orders.info()

complete = orders.loc[
    orders["status"]
    == "Complete",
    ["order_id",
     "channel", "amount"]
]
```

Pandas runs eagerly on one machine: each step executes now, using that machine's memory.

Process columns as a pipeline - not row by row

A clear chain shows the order: **filter** → **clean** → **derive** → **group** → **summarize** → **sort**.

One pandas pipeline

```
summary = (orders
    .loc[orders["status"] == "Complete"]
    .dropna(subset=["amount"])
    .drop_duplicates("order_id")
    .assign(net_amount=lambda d:
        d["amount"] * 0.95)
    .groupby("channel", as_index=False)
    .agg(orders=("order_id", "nunique"),
        revenue=("net_amount", "sum"))
    .sort_values("revenue", ascending=False)
)
```

Read the chain

- 1 Keep complete orders
- 2 Handle missing + duplicates
- 3 Create net_amount

Group by channel
- 5 Count orders + sum revenue
- 6 Put highest revenue first

Join another table

```
orders.merge(customers,
    on="customer_id", how="left")
```

Learn the DataFrame pattern once: Pandas, PySpark, and Polars reuse the same core idea.